

Chain in LangChain

1. What is a Chain in LangChain?

- A Chain in LangChain is a sequence of connected components that work together step-by-step to take an input, process it, and produce a final output.
- Example
Input → Prompt → LLM → Output Parser → Final Output

2. Why do we use Chains in LangChain?

- We use Chains to connect multiple steps of an AI application into a single workflow.
- Main reasons:
 - 1) Connect multiple components – Connect prompt, LLM, output parser, etc.
 - 2) Automate the workflow – Steps execute in the required sequence.
 - 3) Reuse the workflow – Create a chain once and use it multiple times.
 - 4) Simplify code – Instead of calling each component separately, we can connect them together.
 - 5) Build complex applications – Chains can combine multiple AI operations.
- We use chains to connect and organize multiple LangChain components into a single, reusable workflow.

3. Main Types of Chains in LangChain

- 1) Sequential Chain – The output of one step becomes the input of the next step.
 - Example: Generate Content → Summarize Content → Final Output
- 2) Parallel Chain – Multiple independent tasks run at the same time.
 - Example: Generate Title + Generate Description simultaneously.

3) Conditional Chain – Chooses a different path depending on the input or a condition.

- Example:

Question → If Programming → Programming Chain

If Database → Database Chain

4. What is StrOutputParser?

- StrOutputParser is an output parser in LangChain that converts the response received from an LLM into a plain Python string.
- StrOutputParser is used to convert an LLM's response into a simple string (text) format.

```
- from langchain_core.output_parsers import StrOutputParser
```

5. Without Chain

- We manually call and connect every component.

```
from langchain_google_genai import ChatGoogleGenerativeAI
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser

# -----
# 1. Create Gemini Model
# -----

model = ChatGoogleGenerativeAI(
    model="gemini-3.8-flash",
    google_api_key="YOUR_GOOGLE_API_KEY"
)

# -----
# 2. Create Output Parser
# -----

parser = StrOutputParser()
```

```

# -----
# 3. Get Topic From User
# -----

topic = input("Enter topic: ")

# =====
# STEP 1: Generate Blog
# =====

prompt1 = ChatPromptTemplate.from_template(
    "Create an attractive and professional blog about {topic}"
)

# Convert topic into prompt
detailed_prompt = prompt1.invoke({
    "topic": topic
})

# Send prompt to Gemini
response = model.invoke(detailed_prompt)

# Convert AIMessage into plain text
blog_content = parser.invoke(response)

# =====
# STEP 2: Generate Summary
# =====

prompt2 = ChatPromptTemplate.from_template(
    "Create a short and simple 5-line summary of {blog_content}"
)

# Give blog content to the second prompt
detailed_prompt2 = prompt2.invoke({
    "blog_content": blog_content
})

# Send second prompt to Gemini
summary_response = model.invoke(detailed_prompt2)

# Convert response into plain text
summary = parser.invoke(summary_response)

```

```

# =====
# FINAL OUTPUT
# =====

print("\n=====")
print("BLOG")
print("=====")
print(blog_content)

print("\n=====")
print("SUMMARY")
print("=====")
print(summary)

```

1) Sequential Chain in LangChain

- A Sequential Chain is a chain where multiple steps are executed one after another, and the output of one step becomes the input for the next step.
- Simple Flow
Input → Step 1 → Step 2 → Step 3 → Final Output

Example

Suppose we want to create a blog post:

Topic → Generate Title → Generate Content → Create Summary

```

from langchain_google_genai import ChatGoogleGenerativeAI
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser

# Gemini Model
model = ChatGoogleGenerativeAI(
    model="gemini-3.8-flash",
    google_api_key="YOUR_GOOGLE_API_KEY "
)

```

```

# Output Parser
parser = StrOutputParser()

# -----
# STEP 1: Generate Blog Title
# -----

prompt1 = ChatPromptTemplate.from_template(
    "Create an attractive and professional blog title about {topic}"
)

blog_title = prompt1 | model | parser

# -----
# STEP 2: Generate Blog Content
# -----

prompt2 = ChatPromptTemplate.from_template(
    """Write a proper, professional and attractive blog
    based on the following title:

    {title}
    """
)

blog_content = prompt2 | model | parser

# -----
# STEP 3: Generate Blog Summary
# -----

prompt3 = ChatPromptTemplate.from_template(
    """Write a proper and interesting summary
    of the following blog:

    {blog_content}
    """
)

blog_summary = prompt3 | model | parser

# -----

```

```

# USER INPUT
# -----

topic = input("Enter Topic: ")

# -----
# SEQUENTIAL CHAIN
# -----

# Step 1
title = blog_title.invoke({
    "topic": topic
})

# Step 2
content = blog_content.invoke({
    "title": title
})

# Step 3
summary = blog_summary.invoke({
    "blog_content": content
})

# -----
# FINAL OUTPUT
# -----

print("\n=====")
print("BLOG TITLE")
print("=====")
print(title)

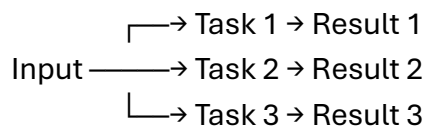
print("\n=====")
print("BLOG CONTENT")
print("=====")
print(content)

print("\n=====")
print("BLOG SUMMARY")
print("=====")
print(summary)

```

2) Parallel Chain in LangChain

- A Parallel Chain is a workflow where multiple independent tasks are executed at the same time using the same input, instead of waiting for one task to finish before starting the next.



- We want to perform three tasks:

1. Generate a blog title
2. Generate keywords
3. Generate a short description

These tasks don't depend on each other, so they can run in parallel.

```
from langchain_google_genai import ChatGoogleGenerativeAI
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser
from langchain_core.runnables import RunnableParallel

# Gemini Model
model = ChatGoogleGenerativeAI(
    model="gemini-3.8-flash",
    google_api_key="YOUR_GOOGLE_API_KEY"
)

parser = StrOutputParser()

# -----
```

```

# TITLE CHAIN
# -----

title_prompt = ChatPromptTemplate.from_template(
    "Create an attractive and professional title about {topic}"
)

title_chain = title_prompt | model | parser

# -----
# KEYWORD CHAIN
# -----

keyword_prompt = ChatPromptTemplate.from_template(
    "Generate 5 important keywords related to {topic}"
)

keyword_chain = keyword_prompt | model | parser

# -----
# DESCRIPTION CHAIN
# -----

description_prompt = ChatPromptTemplate.from_template(
    "Write a short and professional description about {topic}"
)

description_chain = description_prompt | model | parser

# -----
# PARALLEL CHAIN
# -----

parallel_chain = RunnableParallel(
    title=title_chain,
    keywords=keyword_chain,
    description=description_chain
)

# -----
# USER INPUT
# -----

```

```
topic = input("Enter Topic: ")

# Execute Parallel Chain
result = parallel_chain.invoke({
    "topic": topic
})

# -----
# OUTPUT
# -----

print("\n=====")
print("TITLE")
print("=====")
print(result["title"])

print("\n=====")
print("KEYWORDS")
print("=====")
print(result["keywords"])

print("\n=====")
print("DESCRIPTION")
print("=====")
print(result["description"])
```